

Git clone e Git log

Transcrição

Guilherme: Então, essa é a forma que temos para navegar. Vamos fazer uma revisão dos principais comandos?

Começamos com o `git clone`, clonamos o repositório que tínhamos acesso de colaborador.

Paulo: Público sempre podemos clonar, se fosse privado precisaria estar bem configurado. Além de a pessoa te adicionar no time como o Guilherme fez, você precisa ter aquelas chaves para você configurar.



Guilherme: Depois que fizemos o clone, conseguimos ver o log da nossa aplicação de duas formas, `git log` mostrando detalhes de quem é o autor, qual a data em que foi feito, ou o `git log --oneline`, que é o log em apenas uma linha.

Além disso, fizemos o nosso git commit indicando o nome do arquivo que queremos alterar. Podemos até colocar um ponto para pegar o diretório atual e todos de dentro dele: `git commit . -m ""`.

Paulo: Às vezes eu faço. Quem é bom mesmo de Git vai falar para sempre selecionarmos os arquivos exatamente os arquivos que queremos dar commit e suas respectivas mensagens. Geralmente, as pessoas vão acabar usando o ponto e informando qual foi a modificação.

Guilherme: Para enviarmos as nossas modificações: `git push`. Pegamos tudo o que fizemos e mandamos lá para o repositório.

Paulo: Que pode ser mais de um. Podemos empurrar todos os commits que fizemos lá para o repositório origem.

Guilherme: Esses foram os principais comandos. Meu principal desafio para essa aula é: crie um repositório, crie commits dos seus projetos, coloque seus projetos com versões diferentes, crie mensagens de commits que façam sentido para outras pessoas. Às vezes uma mensagem faz muito sentido para mim, mas outra pessoa pode não entender. As mensagens de commit são importantes. Pratiquem os commits locais. Não vimos muitos comandos até agora, mas vimos os essenciais, você vai utilizar mesmo durante seu dia a dia como pessoa desenvolvedora.

Paulo: Esse é o principal do GitHub, seja web ou via linha de comando do Git. Agora vamos trabalhar com os vários *branches*, entender melhor o que é esse *main*, que antigamente até chamava *master*, quais são as opções de podermos trabalhar em paralelo com duas versões e juntar tudo lá no final, o tal do merge. transformaremos em *branches* (galhos) e depois fazemos os merges. Isso é muito frequente, na web vai ficar mais visível quando tiver esses caminhos.

Também veremos o comando `git add`. Se queremos adicionar um arquivo novo não basta fazer commit, temos que informar que o arquivo novo faz parte do diretório.

Veremos mais alguns comandos que são muito importantes.

Tem milhares de comando. Neste curso estamos passando aqueles que são vitais para você trabalhar no seu dia a dia. O Guilherme que manja mais é que vai dar essa parte do curso. E eu volto para finalizar e dar dicas e desafios para você. Chegou nesse momento, você precisa fazer exercícios, precisa praticar.